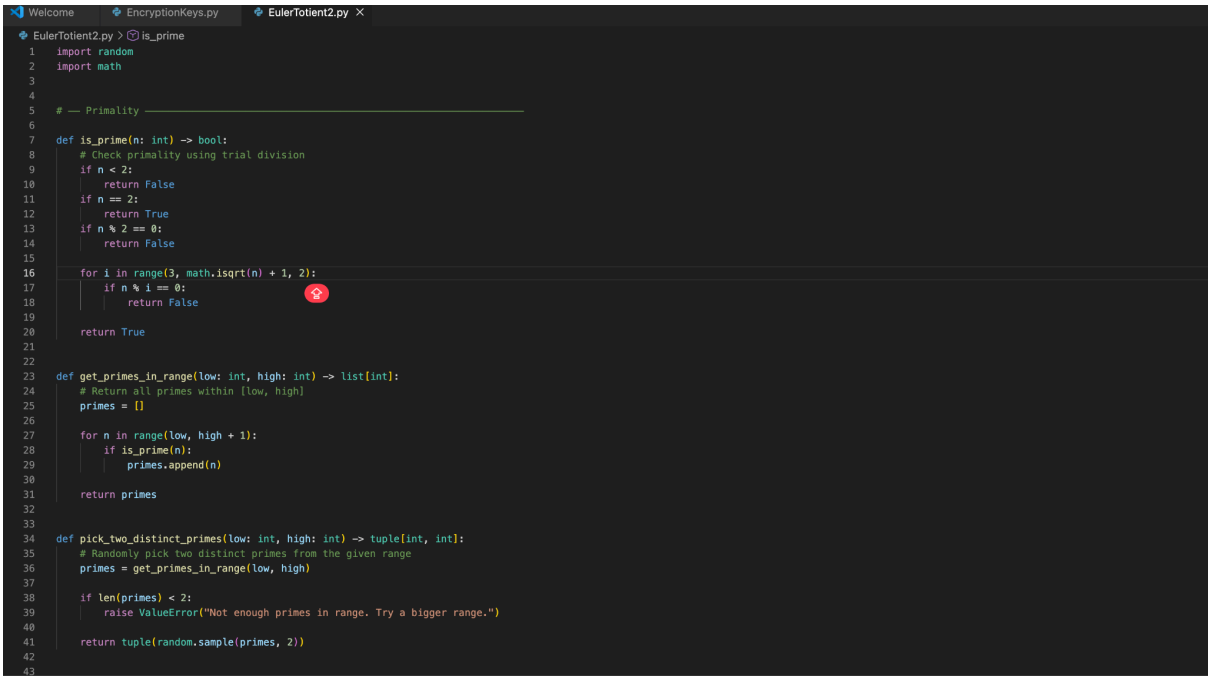
United States International University-Africa

APT3090 CRYPTOGRAPHY AND NETWORK SECURITY

Teddy Baraka Mturi -669791

Lab 1: Key Generation Algorithm Prime numbers



```python
import random
import math


# — Primality ————————————————————————————————

def is_prime(n: int) -> bool:
    # Check primality using trial division
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    for i in range(3, math.isqrt(n) + 1, 2):
        if n % i == 0:
            return False

    return True


def get_primes_in_range(low: int, high: int) -> list[int]:
    # Return all primes within [low, high]
    primes = []

    for n in range(low, high + 1):
        if is_prime(n):
            primes.append(n)

    return primes


def pick_two_distinct_primes(low: int, high: int) -> tuple[int, int]:
    # Randomly pick two distinct primes from the given range
    primes = get_primes_in_range(low, high)

    if len(primes) < 2:
        raise ValueError("Not enough primes in range. Try a bigger range.")

    return tuple(random.sample(primes, 2))
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

Enter lower bound: 20
Enter upper bound: 40

p = 37
q = 29
n = p × q = 1073
```

Trial Division (is_prime / get_primes_in_range)

- Checks whether a number has any divisors other than 1 and itself
- Immediately rejects numbers below 2, and even numbers other than 2
- Tests odd divisors starting at 3, stopping at √n — if n has a factor above √n, a smaller paired factor must exist below it, so checking further is unnecessary
- Returns True only if no divisor is found in that range

Lab 2: Euler's Totient function

Welcome | EncryptionKeys.py | EulerTotient2.py

EulerTotient2.py > is_prime

```python
# ── Euler Totient (Naive) ──────────────────────────────────────

def euler_totient_naive(n: int) -> int:
    # Count how many numbers from 1 to n are coprime with n
    count = 0

    for i in range(1, n + 1):
        if math.gcd(i, n) == 1:
            count += 1

    return count


def coprimes_of(n: int) -> list[int]:
    # Return all numbers in [1, n] that are coprime with n
    coprimes = []

    for i in range(1, n + 1):
        if math.gcd(i, n) == 1:
            coprimes.append(i)

    return coprimes


# ── Display Helper ─────────────────────────────────────────────

def section(title: str) -> None:
    print(f"\n{'─' * 52}")
    print(f"  {title}")
    print(f"{'─' * 52}")


# ── Main Program ───────────────────────────────────────────────

if __name__ == "__main__":

    section("Step 1 — Pick Two Distinct Primes")

    low = int(input("Enter lower bound: "))
    high = int(input("Enter upper bound: "))

    p, q = pick_two_distinct_primes(low, high)

    n = p * q

    print(f"\np = {p}")
    print(f"q = {q}")
    print(f"n = p × q = {n}")

    section("Step 2 — Compute Euler Totient φ(n)")

    phi_n = euler_totient_naive(n)

    print(f"φ({n}) = {phi_n}")

    show = input("\nShow all coprimes of n? (y/n): ").lower()

    if show == "y":
        coprimes = coprimes_of(n)

        print(f"\nNumbers coprime with {n}:")
        print(coprimes)

        print(f"\nCount = {len(coprimes)}")
        print(f"φ({n}) = {phi_n}")
```

Naive Euler Totient (euler_totient_naive / coprimes_of)

- Computes φ(n): the count of integers from 1 to n that share no common factor with n
- Two numbers are coprime when their only shared divisor is 1, i.e. gcd(a, n) = 1
- Iterates through every integer from 1 to n and tests that condition directly using math.gcd
- coprimes_of does the same but returns the actual list instead of a count
- A faster shortcut exists for n = p×q: φ(p×q) = (p−1)(q−1), but the naive method is used here because it works from the definition, making the concept easier to follow